Rust Development

básico

Índice

#	Tema
1	Primitivas
2	Estructuras de control
3	Estructuras de datos
4	Iteradores
5	Funciones
6	Ownership
7	Ejemplo integrado

Primitivas

Tipos básicos integrados al lenguaje, sin overhead de heap allocation y base para construir tipos complejos.

Numéricos

Tipo	Descripción	Ejemplo
`i8, i16, i32, i64, i128, isize`	Enteros con signo (complemento a 2)	`let x: i32 = -42;`
`u8, u16, u32, u64, u128, usize`	Enteros sin signo	`let x: u8 = 255;`
`f32, f64`	Flotantes IEEE 754	`let x: f64 = 3.14;`

Nota: `usize` / `isize` dependen de la arquitectura (64 bits por lo general).

Caracteres

Tipo	Descripción	Ejemplo
`char`	Carácter Unicode (32 bits)	`'a'`, `'ñ'`, `'👩‍💻'`
`str`	String slice (referencia, sin ownership)	`"hola"`
`String`	String con ownership (heap)	`String::from("hola")`

Especiales

Tipo	Descripción	Ejemplo
`bool`	Booleano	`let flag: bool = true;`
`()`	Unit (equivalente a void)	`()`

Colecciones

Tipo	Descripción	Ejemplo
`[T; N]`	Array con longitud fija	`[1, 2, 3]`
`(T, U, ...)`	Tupla con tipos heterogéneos	`(42, "hola", true)`

Estructuras de control

Flujo del programa (secuencia, selección, repetición) donde `if/loop/match` son expresiones y todo bloque retorna un valor.

Bloques (llaves)

```rust
fn main() {

  let resl = {
    let a = 10;
    let b = 5;
    a + b  // última expresión es el valor de retorno
  };

  println!("resultado == {}", resl);
}
```

Condicionales

```
let calificacion = 85;

let mensaje = if calificacion ≥ 90 {
    "Aprobado con honores"
} else if calificacion ≥ 70 {
    "Aprobado"
} else {
    "Reprobado"
};
```

```
let valor = if true { 42 } else { 0 };
```

pattern matching

```
let clima = "lluvia";

match clima {
    "sol"    ⇒ println!("Lleva gafas de sol"),
    "lluvia" ⇒ println!("Lleva paraguas"),
    // El guion bajo es el "comodín" (cualquier otra cosa)
    _        ⇒ println!("Mejor quédate en casa"),
}
```

Ciclos

`loop`

```
let mut tries = 0;

let resl = loop {
    tries += 1;
    if tries == 3 {
        break "Listo después de 3 intentos";
    }
};
```

Ciclos

`while`

```rust
let mut count = 0;

while count < 5 {
    println!("Contando: {}", count);
    count += 1;
}
```

Ciclos

`for`

```
//iterar sobre rangos
for i in 0..=5 { // 0, 1, 2, 3, 4, 5
    print!("{} ", i);
}

// iterar sobre colecciones
let colors = ["rojo", "verde", "azul"];

for color in colors.iter() {
    println!("Color: {}", color);
}

// Iterar con enumerate (índice + valor)
let arr = [10, 20, 30];

for (idx, val) in arr.iter().enumerate() {
    println!("[{}] = {}", idx, val);
}
```

Estructuras de datos

Tipos compuestos para organizar información, mónadas para manejo de ausencia/errores, y collections para grupos de elementos.

`Vec<T>`

Método	Descripción	Ejemplo
`new()`	Crea vector vacío	`Vec::<i32>::new()`
`vec![]`	Macro para inicializar	`vec![1, 2, 3]`
`push()`	Agrega elemento	`v.push(4)`
`pop()`	Retira último elemento	`v.pop()`
`get()`	Acceso seguro con índice	`v.get(0)`
`len()`	Cantidad de elementos	`v.len()`

`Vec<T>` - Ejemplo

```rust
let mut v: Vec<i32> = Vec::new();

v.push(10);
v.push(20);
v.push(30);

println!("Vector: {:?}", v);
println!("Longitud: {}", v.len());

// Acceso seguro con get()
if let Some(elem) = v.get(1) {
    println!("Elemento en [1]: {}", elem);
}
```

`Vec<T>` - Iteración

```rust
let v = vec![10, 20, 30];

for elem in &v {
    println!("{}", elem);
}
```

`HashMap<K,V>`

Método	Descripción	Ejemplo
`new()`	Crea HashMap vacío	`HashMap::<String, i32>::new()`
`insert()`	Agrega par clave-valor	`map.insert("edad", 25)`
`get()`	Obtiene valor por clave	`map.get("edad")`
`contains_key()`	Verifica existencia	`map.contains_key("edad")`

`HashMap` - Ejemplo

```rust
use std::collections::HashMap;

let mut edades = HashMap::new();

edades.insert("Ana", 25);
edades.insert("Luis", 30);

if let Some(edad) = edades.get("Ana") {
    println!("Edad de Ana: {}", edad);
}
```

`HashSet<T>`

Método	Descripción	Ejemplo
`new()`	Crea HashSet vacío	`HashSet::<i32>::new()`
`insert()`	Agrega elemento	`set.insert(1)`
`contains()`	Verifica existencia	`set.contains(&1)`

`HashSet` - Ejemplo

```rust
use std::collections::HashSet;

let mut set: HashSet<i32> = HashSet::new();

set.insert(1);
set.insert(2);
set.insert(3);

println!("Contiene 2: {}", set.contains(&2));
```

Mónadas

Tipos que encapsulan ausencia de valor (`Option`) o posibles errores (`Result`), permitiendo encadenar operaciones de forma segura.

Tipo	Estados	Encadenamiento	Uso principal
`Option<T>`	`Some` , `None`	`.and_then` / `?`	Valor opcional
`Result<T,E>`	`Ok` , `Err`	`.and_then` / `?`	Manejo de errores

Option<T>

```
fn divide(a: f64, b: f64) -> Option<f64> {
  if b == 0.0 { None } else { Some(a / b) }
}

fn main() {
  let a = divide(10.0, 2.0);
  let b = a.and_then(|x| divide(x, 0.0));

  // manejamos la respuesta obtenida
  match b {
    Some(x) => println!("resultado: {}", x),
    None => println!("no hay valor"),
  }
}
```

Result<T, E>

```rust
fn divide(a: f64, b: f64) -> Result<f64, String> {
  if b == 0.0 { Err("error".to_string()) } else { Ok(a / b) }
}

fn main() -> Result<(), String> {
  let a = divide(10.0, 2.0);              // Ok(5.0)
  let b = a.and_then(|x| divide(x, 0.0)); // Err("error")

  // manejamos la respuesta obtenida
  match b {
    Ok(x) => println!("resultado: {}", x),
    Err(e) => println!("error: {}", e),
  }

  Ok(())
}
```

Operador `?`

```
let x = divide(10.0, 2.0)?;    // 5.0
let y = divide(x, 5.0)?;       // 1.0
let resl = divide(y, 0.0)?;    // Err → termina aquí
```

Iteradores

Un iterador produce una secuencia de valores compuesta por fuente (`iter()`), adaptadores que la transforman (`map` , `filter`) y consumidores que producen el resultado final (`collect` , `sum`).

Adaptadores comunes

Método	Descripción	Ejemplo
`map`	Transforma cada elemento	`.map(|x| x * 2)`
`filter`	Filtra elementos	`.filter(|x| x > 0)`
`take(n)`	Toma los primeros n	`.take(3)`
`skip(n)`	Omite los primeros n	`.skip(2)`
`enumerate`	Incluye índice	`.enumerate()`
`zip`	Combina dos iteradores	`.zip(b.iter())`

Consumidores comunes

Método	Descripción	Ejemplo
`collect`	Convierte a colección	`.collect::<Vec<_>>()`
`sum`	Suma todos los elementos	`.sum()`
`fold`	Acumulador personalizado	`.fold(0, |acc, x| acc + x)`
`for_each`	Ejecuta para cada elemento	`.for_each(print!)`

Iteradores - Pipeline básico

```rust
let nums = vec![1, 2, 3, 4, 5];

let result: Vec<i32> = nums.iter()
    .map(|x| x * 2)
    .filter(|x| x > &4)
    .collect();

println!("{:?}", result); // [6, 8, 10]
```

Iteradores - enumerate

```rust
let frutas = vec!["manzana", "pera", "uva"];

for (i, fruta) in frutas.iter().enumerate() {
    println!("[{}] {}", i, fruta);
}
```

Funciones

Fragmentos de código reutilizables que encapsulan lógica, reciben parámetros y pueden retornar valores.

Sintaxis básica

Declaradas con `fn`, reciben tipos anotados y retornan valores explícitos.

```rust
fn greet(name: &str) -> String {
    format!("Hola, {}!", name)
}

fn main() {
    let mensaje = greet("Mundo");
    println!("{}", mensaje);
}
```

Múltiples retornos

Las funciones pueden retornar múltiples valores usando tuplas, útil para operaciones como división y residuo.

```
fn div_mod(a: i32, b: i32) -> (i32, i32) {
    (a / b, a % b)
}

fn main() {
    let (cociente, residuo) = div_mod(10, 3);
    println!("10 = 3 * {} + {}", cociente, residuo);
}
```

Closures

Funciones anónimas definidas inline con sintaxis `|params| expression`, capturan variables de su entorno.

```
// Sintaxis: |params| expression
let doble = |x: i32| x * 2;
let sumar = |a, b| a + b;

println!("{}", doble(5));  // 10
println!("{}", sumar(2, 3)); // 5
```

Closures en contexto

Comúnmente usados con iteradores: `map` transforma y `filter` selecciona elementos según una condición.

```rust
let nums = vec![1, 2, 3, 4, 5];

// filter usa closure con condición
let pares: Vec<&i32> = nums.iter()
    .filter(|x| *x % 2 == 0)
    .collect();
```

Ownership

Sistema único de Rust donde cada valor tiene un único dueño; cuando el dueño sale del ámbito, el valor se libera.

Move semantics

Asignar un valor a otra variable transfiere la propiedad; la variable original deja de ser válida.

```
let v1 = vec![1, 2, 3];
let v2 = v1;              // v1 se "mueve" a v2

println!("{:?}", v2);     // [1, 2, 3]
```

Copy trait

Tipos simples (i32, bool, char) se copian implícitamente; tipos complejos (Vec, String) se mueven.

```
let v = vec![1, 2, 3];  // Vec NO implementa Copy
let w = v;               // se mueve

// println!("{:?}", v); // error: v ya no es válido
println!("{:?}", w);
```

Iteradores y ownership

Método	Retorna	Efecto en ownership
`iter()`	`&T`	Presta referencia inmutable
`iter_mut()`	`&mut T`	Presta referencia mutable
`into_iter()`	`T`	Consume y mueve el valor

```
let nums = vec![1, 2, 3];

for n in &nums     { } // nums sigue válido
for n in &mut nums { } // nums sigue válido
for n in nums       { } // nums ya no existe (movido)
```

Borrowing

Las referencias (`&`) permiten usar un valor sin transferir ownership; la variable original sigue válida.

```rust
fn calcular_largo(s: &String) -> usize {
    s.len()
}

fn main() {
    let s = String::from("hola");
    let largo = calcular_largo(&s); // prestamos s

    println!("'{}' tiene {} caracteres", s, largo);
}
```

Borrowing mutable

Las referencias mutables (`&mut`) permiten modificar un valor prestado sin transferir ownership.

```rust
fn agregar_exclamacion(s: &mut String) {
    s.push('!');
}

fn main() {
    let mut texto = String::from("hola");
    agregar_exclamacion(&mut texto);
    println!("{}", texto); // "hola!"
}
```

Reglas de borrowing

Puedes tener muchas referencias inmutables O una sola mutable, nunca ambas simultáneamente.

Regla	Ejemplo
Puedes tener muchas referencias inmutables	`&x`, `&x`, `&x`
O una sola referencia mutable	`&mut x`
Nunca ambos al mismo tiempo	`&x` + `&mut x` = error

Ownership en funciones

Pasar un valor a una función lo mueve; para usarlo después, se pasa una referencia o se retorna.

```rust
fn procesar(v: Vec<i32>) -> Vec<i32> {
    v.into_iter().map(|x| x * 2).collect()
}

fn main() {
    let nums = vec![1, 2, 3];
    let resultado = procesar(nums); // nums se mueve a la función
    // println!("{:?}", nums); // error
    println!("{:?}", resultado); // [2, 4, 6]
}
```

Ejemplo integrado

Procesar usuarios: filtrar mayores de edad, transformar nombres y registrar resultados aplicando ownership, iteradores, monadas y funciones.

Struct Usuario

```rust
#[derive(Debug)]
struct Usuario {
  nombre: String,
  edad: u8,
}
```

Función de cración de usuarios

```rust
fn crear_usuario(nombre: &str, edad: u8) -> Result<Usuario, String> {
    if nombre.is_empty() {
        Err("El nombre no puede estar vacío".to_string())
    } else if edad > 120 {
        Err("Edad inválida".to_string())
    } else {
        Ok(Usuario {
          nombre: nombre.to_string(),
          edad
        })
    }
}
```

Procesamiento completo

```rust
fn main() -> Result<(), String> {
    let nombres = vec!["Ana", "Luis", "María"];
    let edades = vec![25, 17, 45];

    // Iteradores + Funciones + Monadas
    let usuarios: Vec<Usuario> = nombres
        .iter()
        .zip(edades.iter())
        .map(|(n, e)| crear_usuario(n, *e))
        .collect::<Result<Vec<_>, _>>()?;

    // Iteradores + Closures + Ownership: filtrar y transformar
    let resultado: Vec<String> = usuarios
        .into_iter()                        // Ownership: consume usuarios
        .filter(|u| u.edad ≥ 18)            // Closure: filtrar
        .map(|u| u.nombre.to_uppercase())   // Closure: transformar
        .collect();

    println!("Mayores de edad: {:?}", resultado);
    Ok(())
}
```

Conceptos aplicados

Concepto	Aplicación
Funciones	`crear_usuario()` combina datos y valida
Monadas	`Result<T, E>` con `?` encadena errores
Iteradores	`iter()`, `zip()`, `map()`, `filter()`, `collect()`
Closures	`|u| u.edad ≥ 18`, `|u| u.nombre ...`
Ownership	`into_iter()` consume `usuarios`